Final

May 3, 2021

[1]:
```
# written in Julia 1.6.0 by OIHernandez as Final backup for
# MATH F615 Numerical Analysis of Diff Eqs by DAMaxwell
# https://github.com/mathemonads/HW_615/blob/main/Final.ipynb

# import Pkg; Pkg.add("ShiftedArrays")
ENV["PYTHON"] = "python3"
```

[1]:
```
"python3"
```

[2]:
```
using BandedMatrices
using LinearAlgebra
using DataFrames
using Distances
using GLM
using LaTeXStrings
using NLsolve
using Roots
using ShiftedArrays
# set plotting environment
using PyCall
using Plots; pyplot()

dim  = (900,400)
rd(x)= round(x, digits=2)

# previous
function newton(f, Df, x0; tol=1e-8, nmax=1e8)
    f0 = maximum(abs.(f(x0)))
    xk = x0
    err = 1
    d = Df(xk)
    z = f(xk)
    n = 0
    xp = xk
    while (n<nmax && err > tol)
        n += 1
        p = -d\z
        xp = xk + p
```

```
        # terminate whenever error is less than specified tolerance or fraction
 ↪of initial value m
        err = maximum(abs.(f(xp)))
        if err < tol # err < abs(err/(err-f0)^n)
            return (n,xp)
        end
        xk = xp
        d = Df(xk)
        z = f(xk)
    end
    return (n, xp)
end

# function test(T, S, u0, h, k, solver)
#     N = Int(round(S/h-1))
#     M = Int(round(T/k))
#     s0 = 0.0
#     t0 = 0.0
#     x,t,u = solver(a, b, u0, t0, s0, T, S, M, N, k, h)
#     u_exact = zeros(length(t), length(x))
#     for j in 1:length(t)
#         for i in 1:length(x)
#             u_exact[j,i] = u_sol(x[i], t[j])
#         end
#     end
#     e = u_exact-u
#     return x, t, u, u_exact, e, maximum(abs.(e[length(t),:])), a*k/h
# end

function regress_convergence(h, Z)
    df = DataFrame(h=h, Z=Z)
    fm = @formula(log(Z) ~ 1 + log(h)) # log(Z) = log(C)*1 + p*log(h)
    lr = lm(fm, df) # fit a straight line to the data
    lC, p = GLM.coef(lr) # retreive constant coefficients log(C) and p
    C = exp(lC)
    return p, C
end
```

[2]: regress_convergence (generic function with 1 method)

1 Newton's method

Consider the problem

$$u_{xx} + \gamma u^4 = f(x)$$

on the interval $0 \leq x \leq 1$ with $u(0) = u(1) = 0$.

1.1 Example problem

If $u(x) = sin(3\pi x)$, then the value of f is given by:

$$\begin{aligned} f(x) &= \frac{d}{dx}\left(-3\pi\cos(3\pi x)\right) + \gamma\left(\sin(3\pi x)\right)^4 \\ &= 9\pi^2\sin(3\pi x) + \gamma\sin^4(3\pi x) \end{aligned}$$

1.2 Code

1.2.1 Approximation

$$\begin{aligned} \frac{u(x_{i+1}) - 2u(x_i) + u(x_{i-1})}{h^2} + \gamma u(x_i)^4 &= f(x_i) \\ \frac{u_{i+1}}{h^2} + \frac{-2u_i}{h^2} + \frac{ui-1}{h^2} + \gamma u_i^4 \gamma u_i^4 &\approx f_i \\ F(u) := A\vec{u} + \gamma\vec{u}^4 - \vec{f} &= 0, \end{aligned}$$

where A is $\frac{-2}{h^2}$ on the diagonal and $\frac{1}{h^2}$ above and below the diagonal

[3]:
```
function solve(f, t0, u0, T, M, gamma)
    h = (T-t0)/M
    t = [t0:h:T;]
    u = zeros(length(t))
    u[length(t)] = u0
    u[1] = u0
    u[2] = u0
    N = length(u[2:length(t)-1])
    l = 1.0/h^2
    A = BandedMatrix((-1=>repeat([l],outer=[N-1]), 0=> repeat([-2*l],outer=[N]),
 ↪1=> repeat([l],outer=[N-1])), (N,N))
    F(u_curr) = Matrix(A)*u_curr +gamma*[uk^4 for uk in u_curr] - [f(t[k]) for k
 ↪in 1:length(u_curr)]
    Df(u_curr) = A + BandedMatrix(0=>[4*gamma*uk^3 for uk in u_curr])
    n, u[2:length(t)-1] = newton(F, Df, u[2:length(t)-1], tol=1e-9)
    return n, t, u
end

t0 = 0.0
u0 = 0.0
T = 1.0

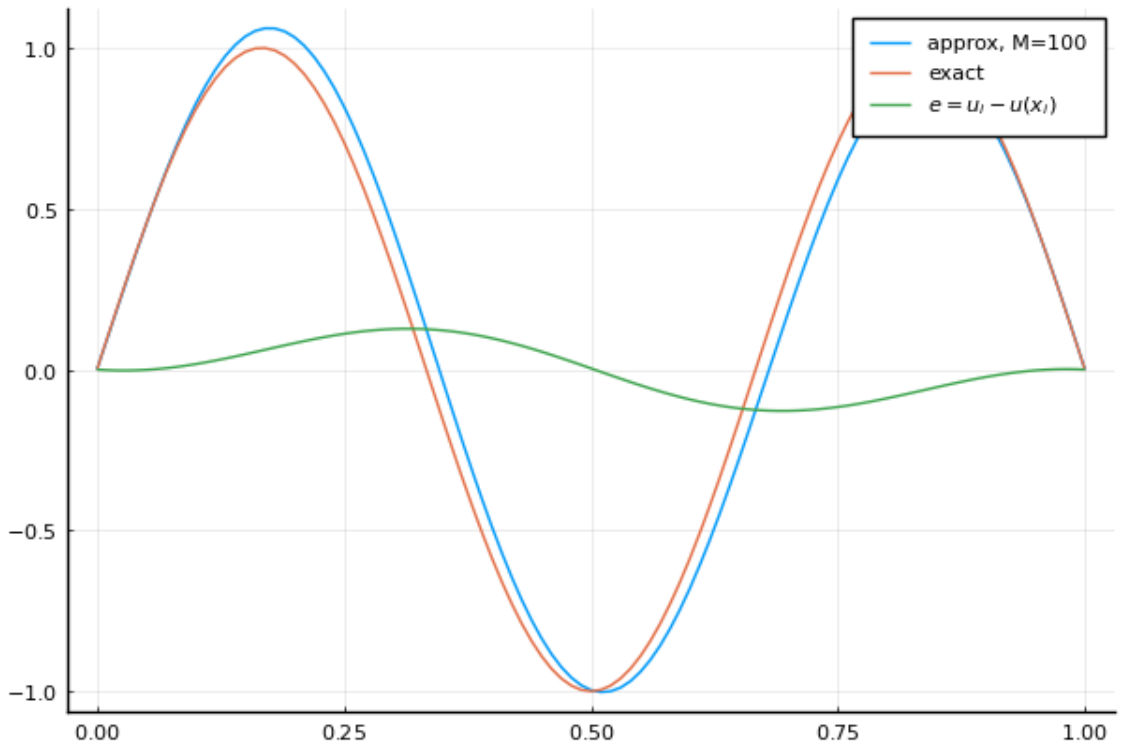
u1_exact(x) = sin(3*pi*x)
function rhs(gamma)
    f(x) = gamma*sin(3*pi*x)^4 -9*sin(3*pi*x)*pi^2
    return f
end
L"f(x) = \gamma\sin^4(3\pi x) - 9\sin(3\pi x)\pi^2"
```

[3]: $f(x) = \gamma \sin^4(3\pi x) - 9\sin(3\pi x)\pi^2$

[4]:
```
M = 100 # 1000 is unreasonably good

gamma = 0.0
n1, t1, u1 = solve(rhs(gamma), t0, u0, T, M, gamma)
u1_e = [u1_exact(tk) for tk in t1]

plot(t1,u1, label="approx, M=$M")
plot!(t1, u1_e, label="exact")
plot!(t1, u1-u1_e, label=L"e=u_i-u(x_i)")
```

[4]:


[5]:
```
function get_conv(Ms, gamma)
    println(Ms)
    h = zeros(length(Ms))
    Z = zeros(length(Ms))
    for i in 1:length(Ms)
        M = Int(round(Ms[i]))
        h[i] = (T-t0)/M
        n, t, u = solve(rhs(gamma), t0, u0, T, M, gamma)
#         u_e = [u1_exact(tk) for tk in t]
#         e = u-u_e
#         eM = abs.(e[M])
```

```
#         Z[i] = eM
    end
    return h, Z
end

gamma = 0.0
# Ms = [4:50:100;]
Ms = [500:100:1200;]
h, Z = get_conv(Ms, gamma)
# p, C = regress_convergence(h, Z)
# println(p)
# O(dt) = C*dt^p
# plot(O, minimum(h), maximum(h), label=L"O(dt)=%$C\cdot (dt)^{%$p}", xaxis=:
 ↪log, yaxis=:log, xlabel="dt", ylabel="|e|")
# scatter!(h, Z, legend=:bottomright, label="(dt, |e|)")
```

```
[500, 600, 700, 800, 900, 1000, 1100, 1200]
```

[5]:
```
([0.002, 0.0016666666666666668, 0.0014285714285714286, 0.00125,
0.0011111111111111111, 0.001, 0.0009090909090909091, 0.0008333333333333334],
[0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0])
```

Indeed we observe quadratic orders of convergence (~1.93, 1.94) when $\gamma = 0, 1$, respectively.

2 Klein-Gordon

Consider the Klein-Gordon equation

$$c^2 u_{xx} = u_{tt} + bu, \qquad b, c > 0,$$

where the initial and boundary conditions are given by $u(x,0) = f(x)$, $u_t(x,0) = g(x) = 0$, and $u(0,t) = u(l,t) = 0$, which yields the following equation solution:

$$u(x,t) = \sum_{n=1}^{\infty} a_n \sin(\lambda_n x) \cos\left(t\sqrt{b + \lambda_n^2 c^2}\right),$$

where $\lambda_n = n\pi$ and $a_n = 2\int_0^1 f(x)\sin(\lambda_n x)\,dx$.

2.1 FDM

We obtain

$$c^2 \frac{u_{i+1,j} - 2u_{i,j} + u_{i-1,j}}{h^2} = \frac{u_{i,j+1} - 2u_{i,j} + u_{i,j-1}}{k^2} + bu_{i,j}$$

$$u_{i,j+1} = bk^2 u_{i,j} - \frac{c^2 k^2}{h^2}\left(u_{i+1,j} - 2u_{i,j} + u_{i-1,j}\right) - 2u_{i,j} + u_{i,j-1},$$

with $u_{i,0} = f_i = f(x_i)$, $u_{0,j} = u_{N,j} = 0$, and $\frac{u_{i,1} - u_{i,-1}}{2k} = g_i = 0$. This yields a ghost point $u_{i,-1} = u_{i,1}$, or equivalently states that $u_{i,0} = u_{i,2}$.

This method is quadratic in space and time. Since it has the same analytic and numerical domains of dependence as the wave equation, this method satisfies the CFL condition.

2.2 Stability

For stability, we also require that $\frac{k^2}{h^2} \leq \frac{4}{4+h^2}$ ## Dispersivity/dissipativity

Suppose $u(x,t) = e^{I(kx-wt)}$. Then, the positivity of b, c, k imply that

$$\begin{aligned} 0 &= (Ik)^2 c^2 e^{I(kx-wt)} - (Iw)^2 e^{I(kx-wt)} - b e^{I(kx-wt)} \\ 0 &= -k^2c^2 + w^2 - b \\ w_r + 0 = w &= +\sqrt{k^2c^2 + b}, \end{aligned}$$

and thus that $w_i = 0 \leq 0$, so the equation is stable and non-dissipative.

Since $v_{ph} = \frac{w_r}{k} = \sqrt{c^2 + \frac{b}{k^2}}$ depends on k, the scheme is dispersive.

3 TR-BDF2

THe TR-BDF2 method is an implicit second-order Runge-Kutta method of the following form:

$$\begin{aligned} u_\star &= u_n + \frac{k}{4}\left[f(u_n) + f(u_\star)\right] \\ u_{n+1} &= \frac{1}{3}\left[4u_\star - u_n + kf(u_{n+1})\right] \end{aligned}$$

3.1 L-stability

Its A-stability follows from that of its component methods. Similarly, it satisfies $\lim_{z\to-\infty} R(z) = 0$ because the limit of the stability polynomials of each of the component methods is 0. Alternative, its stability polynomial

$$R(z) = \frac{1}{12}\left(\frac{12+5z}{12-7z+z^2}\right)$$

can be directly seen to approach 0 as z approaches $-\infty$.

3.2 Numerical code

[6]:
```
function gen_diags(N, kappa, theta, lambda)
    A = BandedMatrix((-1=>repeat([kappa*theta*lambda],outer=[N-1]),␣
 ↪0=>repeat([1-2*kappa*theta*lambda],outer=[N]) ,␣
 ↪1=>repeat([kappa*theta*lambda],outer=[N-1])), (N,N))
    B = BandedMatrix((-1=>repeat([-(1-theta)*kappa*lambda],outer=[N-1]),␣
 ↪0=>repeat([1+2*(1-theta)*kappa*lambda],outer=[N]) ,␣
 ↪1=>repeat([-(1-theta)*kappa*lambda],outer=[N-1])), (N,N))
    return A, B
end
```

```
function forcedheat(u0, T, M, N, theta, kappa)
    s0 = 0.0
    S  = 1.0
    t0 = 0.0
    h  = S/(N+1)
    k  = T/M
    lambda = k/h^2
    A, B = gen_diags(N, kappa, theta, lambda)
    succ(u_curr) = B\(A*u_curr) # efficient linear solver since matrices are␣
 ↪tridiagonal
    return theta_method(u0, T, M, N, theta, succ, kappa) # BE if theta = 0.0
end

function theta_method(u0, T, M, N, theta, succ, kappa)
    h = 1/(N+1)
    k = T/M
    x = LinRange(0, 1, N+2)
    x_int = x[2:N+1]
    t = LinRange(0, T, M+1)
    lambda = k/h^2

    u = zeros(length(t), length(x))
    u_curr = [u0(x_int[i]) for i in 1:length(x_int)]
    u[1,2:N+1] = u_curr

    A, B = gen_diags(N, kappa, theta, lambda)
    A0, B0 = gen_diags(N, kappa, 0.0, lambda)
    A05, B05 = gen_diags(N, kappa, 0.5, lambda)
    C = BandedMatrix((-1=>repeat([kappa*1.0*lambda],outer=[N-1]),␣
 ↪0=>repeat([-2*kappa*1.0*lambda],outer=[N]) , 1=>repeat([kappa*1.
 ↪0*lambda],outer=[N-1])), (N,N))
    B0 = I- C .* k/3

    for j in range(2, stop=M+1)
        u_star = B05\(A05*u_curr) # trapezoidal
        u_curr = B0\(4*u_star-u_curr) ./ 3
        u_curr = u_star
        u[j,2:N+1] = u_curr
    end
    return (x, t, u)
end

f(x,t) = 0
H = [0.05, 0.02, 0.01, 0.005, 0.002, 0.001]
T = 0.1
kappa = 1.0/20 #1.0/20
J = 5
```

```
u3_0(x) = sin(J*pi*x)
u_sol(t,x) = exp(-kappa*t*(J*pi)^2)*sin(5*pi*x)

hb = H[5]
kb = 2*hb

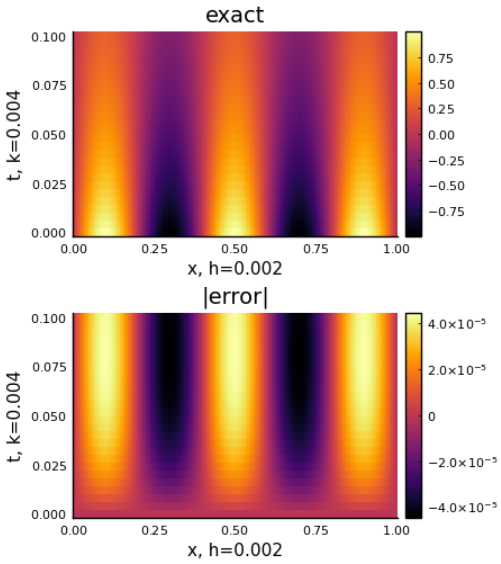
function sol_error(f, u0, kappa, h, k, u_sol, theta)
    S = 1.0
    T = 0.1
    N = Int(round(S/h))
    M = Int(round(T/k))
    x, t, u = forcedheat(u0, T, M, N, theta, kappa)
    u_exact = zeros(length(t), length(x))
    e = zeros(length(t), length(x))
    e_inf = -Inf
    for j in 1:length(t)
        for i in 1:length(x)
            u_exact[j,i] = u_sol(t[j], x[i])
            e[j,i] =  u_exact[j,i] - u[j,i]
            e_inf = max(e_inf, abs(e[j,i]))
        end
    end
    return x, t, u, u_exact, k/(h^2), e, e_inf, e[length(t),:]
end

theta = 0.5
x3, t3, u3, u3_exact, l3, e3, e3_inf, e3T = sol_error(f, u3_0, kappa, hb, kb,␣
 ↪u_sol, theta)
# maximum(abs.(e3T))

function hm(u, title)
    return heatmap(x3, t3, u, title=title, xlabel="x, h=$hb", ylabel="t, k=$kb")
end

plot(hm(u3_exact, "exact"), hm(u3, "approx"), hm(e3, "|error|"), size=(1000,500))
```

[6]:



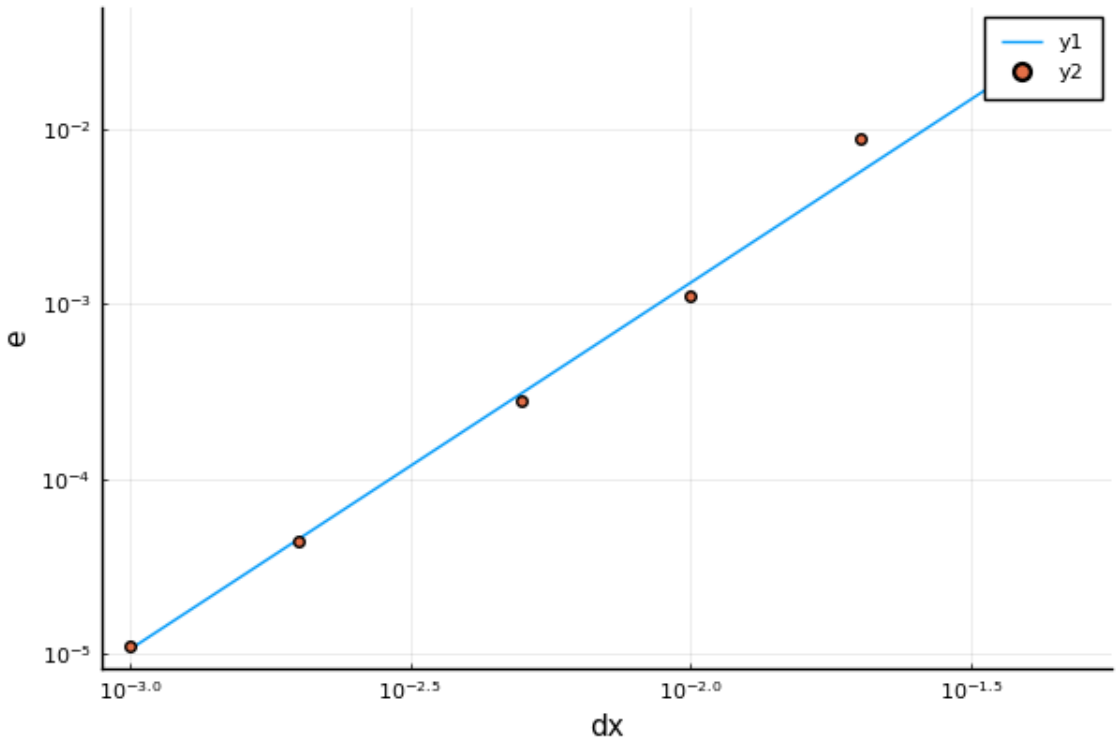
[7]:
```
function heat_error(theta)
    Z = zeros(length(H))
    for c in 1:length(H)
        xc, tc, uc, uc_exact, lc, ec, ec_inf, ec_T = sol_error(f, u3_0, kappa,␣
 ↪H[c], 2*H[c], u_sol, theta)
        e = ec_inf
        Z[c] = maximum(abs.(e))
    end
    return Z
end

function gen_errors()
    theta = 0.5
    return heat_error(theta)
end

Zc = gen_errors()
pc, Cc = regress_convergence(H, Zc)
println((pc, H, Zc))
O(h) = Cc*h^pc
plot(O, minimum(H), maximum(H), xlabel="dx", ylabel="e", xaxis=:log, yaxis=:log)
scatter!(H, Zc)
```

```
(2.0977685615415704, [0.05, 0.02, 0.01, 0.005, 0.002, 0.001],
[0.03216891105208841, 0.008822945448453245, 0.0011293316081060212,
0.00027962466966491295, 4.452284818617924e-5, 1.1114534765155337e-5])
```

[7]:



3.3 Merits

The TR-BDF2 method converges faster than the Backwards Euler method at the cost of additional computation, and it does not produce the same oscillatory errors as the Crank-Nicolson method does.

4 Finite Volumes

[8]:
```
function forward_euler(f, t0, u0, T, M)
    h = (T-t0)/M
    t = [t0:h:T-h;]
    u = [[0.0, 0.0] for i=1:M]
    u[1] = u0
    u_curr = u0

    for k in range(1, stop=M-1)
        u_curr = u_curr + h * f(t[k], u_curr)
        u[k+1] = u_curr
    end
    return (t,u)
end
```

```
function rk4(f, t0, u0, T, M)
    h = (T-t0)/M
    t = [t0:h:T-h;]
    u = [[0.0, 0.0] for i=1:M]
    u[1] = u0
    u_curr = u0

    for k in range(1, stop=M-1)
        k1 = h*f(t[k], u_curr)
        k2 = h*f(t[k]+h/2, u_curr+k1/2)
        k3 = h*f(t[k]+h/2, u_curr+k2/2)
        k4 = h*f(t[k+1]+h, u_curr+k3)
        u_curr = u_curr + (1/6) * (k1+2*k2+2*k3+k4)
        u[k+1] = u_curr
    end
    return (t,u)
end

function rk2(f, t0, u0, T, M)
    h = (T-t0)/M
    t = [t0:h:T-h;]
    u = [[0.0, 0.0] for i=1:M]
    u[1] = u0
    u_curr = u0

    for k in range(1, stop=M-1)
        k1 = h*f(t[k], u_curr)
        k2 = h*f(t[k]+h/2, u_curr+k1/2)
        u_curr = u_curr + k2
        u[k+1] = u_curr
    end
    return (t,u)
end

function sol_error(exact_sol, approx_method, f, t0, u0, T, M)
    approx = approx_method(f, t0, u0, T, M)
    t, sol = approx
    e_inf = 0 # maximum error observed
    for k in range(1, stop=length(t))
        ek = abs.(exact_sol(t[k])- sol[k])
        ek = sum(ek)
        e_inf = max(e_inf, ek)
    end
    return e_inf
end

A = [0 -1; 1 0]
```

```
f(t,u) = A*u
t0 = 0
u0 = [1, 0]
T = 1
M = 10

function sol(t)
    x(t) = cos(t)
    y(t) = sin(t)
    return [x(t), y(t)]
end

h = [10.0^(o) for o=-2.7:0.01:-2.4]
Z = Dict(
    "forward_euler" => zeros(Float64, length(h)),
    "rk2" => zeros(Float64, length(h)),
    "rk4" => zeros(Float64, length(h)),
)

for k in range(1, stop=length(h))
    M = round(Int, (T-t0)/h[k])
    Z["forward_euler"][k] = sol_error(sol, forward_euler, f, t0, u0, T, M)
    Z["rk2"][k] = sol_error(sol, rk2, f, t0, u0, T, M)
    Z["rk4"][k] = sol_error(sol, rk4, f, t0, u0, T, M)
end

for (k,Zv) in Z
    p, M = regress_convergence(h, Z[k])
    println(k, ": O(Mh^p) for (p,M)=", (p, M))
end
```

```
forward_euler: O(Mh^p) for (p,M)=(1.0003092093827588, 0.6913349312723713)
rk4: O(Mh^p) for (p,M)=(4.003309620864294, 0.011720606097642734)
rk2: O(Mh^p) for (p,M)=(2.0011070547975933, 0.23136564943398963)
```

[9]:
```
function upwind_flux(u,a)
    F = zeros(length(u)+1)
    if a >= 0
        F[1:length(u)] = a .* u
        F[length(u)+1] = 0
    elseif a<0
        F = circshift(a .* u, length(t)-1)
    end
    return F, circshift(F, 1)
end

N = 4
```

```
u0 = [1,2,3]
a = 2

upwind_flux(u0, a)
```

[9]:
```
([2.0, 4.0, 6.0, 0.0], [0.0, 2.0, 4.0, 6.0])
```

[10]:
```
function advection(u0,a,T,N,M, ode_solver,flux_computer)
# function advection(u0, t0, s0, T, S, M, N, a, A)
    s0, t0, S = 0.0, 0.0, 1.0
    h = S/(N+1)
    k = T/M
    t = Array(LinRange(t0, Float64(T), M+1))
    x = Array(LinRange(s0, S, N+2))
    x_int = x
    u = zeros(length(t), length(x))
    u[1,:] = [u0(x[i]) for i in 1:length(x)]

#     succ(u_curr) = A*u_curr
#     u[1,:] = [u0(xi) for xi in x_int]
#     u[2,1:N+2] = u[1,:] + A*((1/2)*A*u[1,:])
#     println(x)
    u_curr = u[1,:]
#     println(u_curr)
    for j in range(2, stop=M+1)
        Fnext, Fprev = flux_computer(u_curr[1:length(u_curr)-1], a)
#         println(Fnext)
#         println(Fprev)
#         println((Fnext-Fprev) .* k/h)
        u_curr = u_curr + (Fnext-Fprev) .* k/h
#         u_prev = u[j-1,:]
#         u_curr = succ(u_prev)
        u[j,:] = u_curr

    end
    return (x, t, u)
end

a = 1.0
u4_0(x) = sin(2*pi*x)
u4_e(t,x) = sin(2*pi*(x-t))

function test_fv(exact_sol, t0, u0, T, M, N)
    x, t, u = advection(u4_0,a,T,N,M, forward_euler, upwind_flux)
    e_inf = -Inf # maximum error observed
    e = zeros(length(t), length(x))
    u_exact = zeros(length(t), length(x))
```

```
    for k in range(1, stop=length(t))
        for i in 1:length(x)
            u_exact[k,i] = exact_sol(t[k], x[i])
            ek = u_exact[k,i]- u[k,i]
            e[k,i] = ek
        end
    end
    return t, x, u, u_exact, e
end

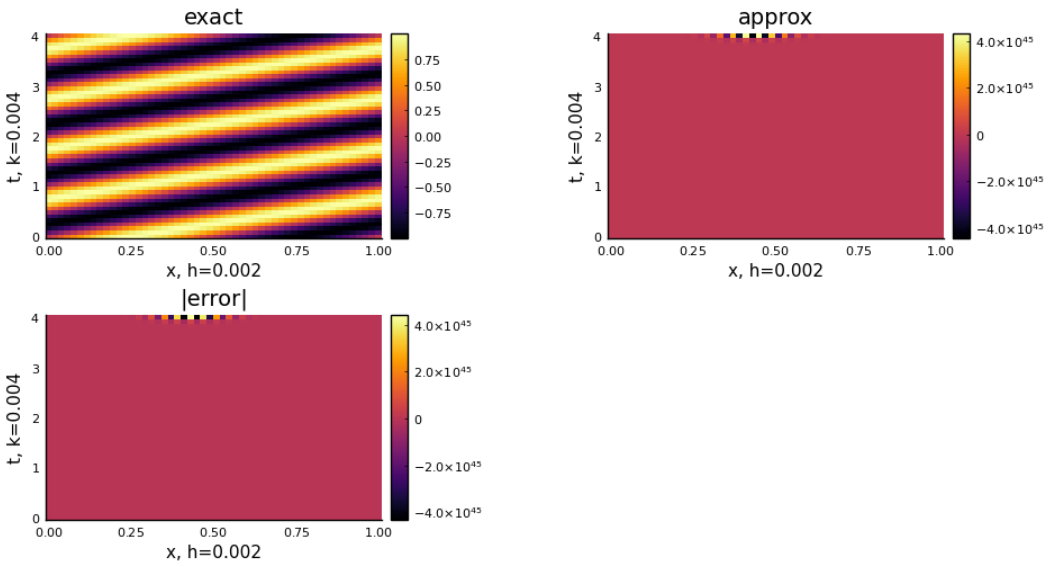

T = 4
M = 50
N = M
t4, x4, u4, u4_exact, e4 = test_fv(u4_e, t0, u4_0, T, M, N)

function hm4(u, title)
    return heatmap(x4, t4, u, title=title, xlabel="x, h=$hb", ylabel="t, k=$kb")
end

println(sum(abs.(e4[2,:])))
plot(hm4(u4_exact, "exact"), hm4(u4, "approx"), hm4(e4, "|error|"),␣
 ↪size=(1000,500))
```

```
32.785930789580334
```

[10]:



I have not yet been able to get the method to converge. I suspect this is due to a mistake in the upwind flux computation as it relates to the periodic boundary condition.

[11]:
```
# EOF
```